QML Lecture · 2 일차

실습 설명 자료

segmentation 전처리 → 양자 회로 → QGAN 학습

오리엔테이션

2 일차 진행 순서

노트북 섹션 순서대로 실습하고 , 각 블록 앞에 해당 설명 PART 를 진행함

§ 0–2
환경 · 데이터
Colab 설정
HAM10000 EDA
PART 0

→

§ 3
전처리
segmentation
K-means + FCM
16×16 축소
PART 1

→

§ 4
양자 회로
회로 1 개 구성
출력 확인
PART 2

→

§ 5
QGAN 학습
Colab 약 35 분
(400 step)
PART 3

→

§ 6
생성 · 평가
지표 4 종
PART 4

→

§ 7
정리
Drive 저장

운영 포인트 § 5 학습 셀을 먼저 실행한 뒤 PART 3 을 설명함 - 학습이 도는 약 35 분이 설명 시간이 됨 . 진행 로그는 20 step 마다 출력됨 .

PART 0

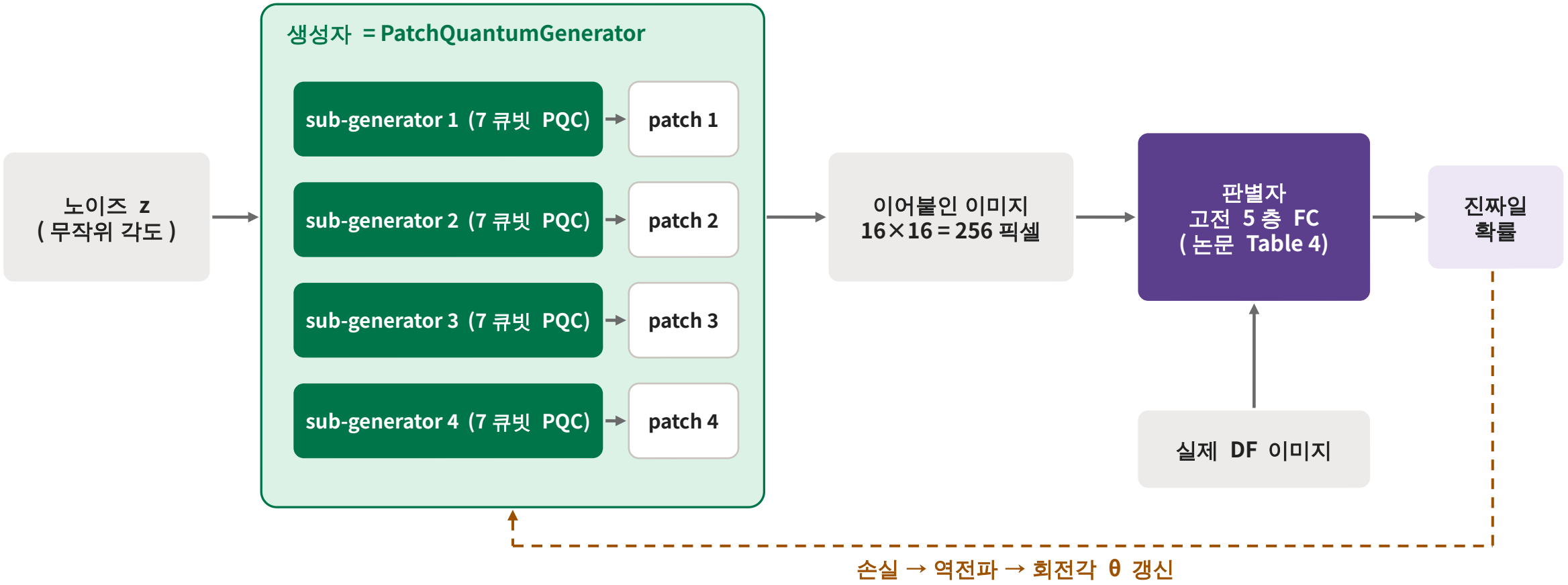
1 일차 복습

노트북을 시작하기 전 - 오늘 구현하는 QGAN 의 구조와 논문 대비 축소 범위

오늘 구현하는 것	논문 ① QGAN 의 patch 방식 생성자 - sub-generator 4 개 + 고전 판별자
입력 데이터	HAM10000 DF 115 장 → segmentation → 16×16 그레이스케일
오늘의 결과물	직접 학습한 생성자 checkpoint + 생성품질 지표 4 종 표 (3 일차 비교 기준)

QGAN 구조 - 생성자만 양자회로

patch 방식 : sub-generator 마다 이미지 조각 (patch) 을 하나씩 만들고 이어붙임



PART 0 · 1 일차 복습

논문과 오늘 실습의 차이

회로의 형태 (RY 인코딩 + RY·CZ 변분층 + post-selection) 는 같고 , 규모와 학습량을 줄임

항목	논문 (Andra et al.)	2 일차 실습	이유
sub-generator	16 개	4 개	2·4·8·16 개 실측 중 품질 최고
큐빗 (sub-generator 당)	데이터 7 + ancilla 1	데이터 6 + ancilla 1	$256 = 4 \times 2^6$
회로 depth	10	10	논문과 동일
판별자	5 층 FC (Table 4)	5 층 FC (Table 4)	논문과 동일
학습량	2,000 epoch (1 ~ 1.5 시간)	400 step (Colab 약 35 분)	수업 시간 안에 완료
실행 환경	GPU	Colab CPU, exact 시뮬레이션	설치 · 계정 불필요

참고 sub-generator 16 개 (논문 설정) 는 판별자가 이겨 loss_d 가 0.003 까지 떨어짐 - 용량을 늘린다고 품질이 오르지 않음

PART 1 · 노트북 §2 – §3 직전

데이터와 segmentation 전처리

HAM10000 에서 DF·NV 를 고르고 , 병변만 남겨 16×16 으로 줄이는 과정

이 파트의 코드	segment_fcm_kmeans · resize_for_encoding · preprocess_class
핵심 질문	왜 병변만 남기는가 / 왜 16×16 인가 / 축소는 어떻게 하는가

범위 축소 - DF vs NV

7 개 클래스 중 가장 적은 DF 를 증강 대상으로 , 가장 많은 NV 를 비교 대상으로 둠

10,015
전체 이미지

115
DF (피부섬유종)

6,705
NV (모반)

1 : 58
DF : NV

```
metadata = pd.read_csv(f"{DATA_DIR}/HAM10000_metadata.csv")
counts = metadata["dx"].value_counts()

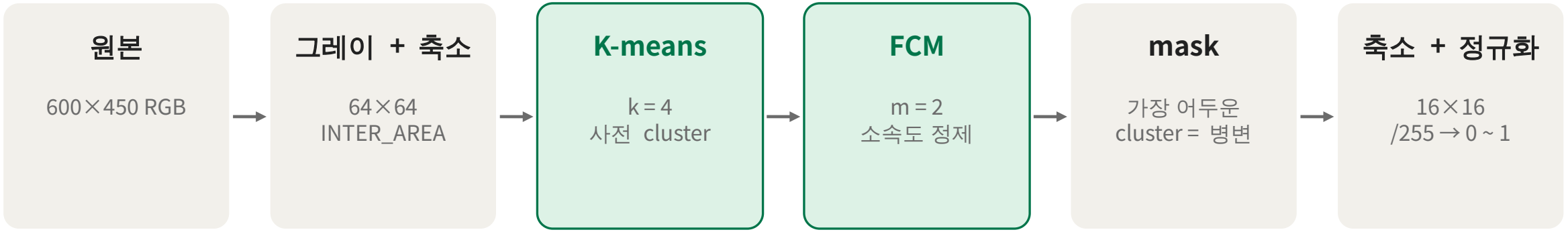
# df     115   <-  증강 대상
# nv    6705   <-  비교 대상
```

읽을 점

- metadata 는 image_id 와 진단명 (dx) 을 연결하는 표임
- 이미지 파일은 part_1·part_2 두 폴더에 나뉘어 있어 find_image_path 가 양쪽을 확인함

전처리 pipeline

원본 → 병변만 남긴 64×64 → 16×16, 0 ~ 1. 저장소 자산 (df_preprocessed.npy) 도 같은 조건으로 만들어짐



논문 Section 3.1 의 K-means + Fuzzy C-means hybrid segmentation

왜 병변만 남기는가

논문 Table 10: raw 이미지로 증강하면 분류 성능이 오히려 떨어지고 , segmentation 을 거친 이미지에서만 개선됨

왜 64×64 를 거치는가

FCM 비용이 픽셀 수에 비례함 . 원본 27 만 픽셀 대신 4,096 픽셀로 클러스터링해 1 장당 수 초로 줄임

segment_fcm_kmeans 읽기

밝기 1 채널만 써서 4 개 cluster 로 나누고 , 가장 어두운 cluster 를 남김

```
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
gray = cv2.resize(gray, (64, 64), interpolation=cv2.INTER_AREA)
pixels = gray.reshape(-1, 1).astype(np.float32)      # 1

km = KMeans(n_clusters=4, n_init=4, random_state=0)  # 2
km = km.fit(pixels)
dist = np.abs(pixels - km.cluster_centers_.reshape(1, -1))
u_init = 1.0 / (dist + 1e-6) ** 2                    # 3
u_init = (u_init / u_init.sum(axis=1, keepdims=True)).T

cntr, u, *_ = fuzz.cluster.cmeans(pixels.T, c=4, m=2.0,
                error=1e-4, maxiter=100, init=u_init)
membership = np.argmax(u, axis=0)                    # 4
lesion = np.argmin(cntr.flatten())
mask = (membership == lesion).reshape(64, 64) * 255
return cv2.bitwise_and(gray, gray, mask=mask)        # 5
```

1. **밝기만 사용**
   (픽셀 수 , 1) - 위치 정보 없이 밝기로 cluster 를 나눔
2. **K-means 사전 cluster**
   k-means++ 초기화를 4 번 시도해 가장 좋은 결과 채택
3. **FCM 초기 소속도**
   중심까지 거리의 역제곱 → 빠르고 재현 가능한 수렴
4. **병변 cluster 선택**
   소속도 최대 cluster 로 라벨 , 가장 어두운 중심 = 병변
5. **mask 적용**
   병변은 원래 밝기 유지 , 배경은 0

K-means 와 Fuzzy C-means

K-means 는 픽셀마다 cluster 하나 , FCM 은 cluster 별 소속도를 나눠 가짐

	K-means	Fuzzy C-means (FCM)
배정 방식	hard - 픽셀마다 cluster 1 개	soft - 소속도 u (픽셀별 합 = 1)
중심 갱신	배정된 픽셀의 평균	소속도 ^m 가중 평균
주요 파라미터	n_clusters, n_init	c, m (fuzziness), error, maxiter, init
이 코드에서의 역할	FCM 초기값을 만드는 사전 cluster	소속도 정제 → argmax 로 최종 라벨
출력	cluster_centers_, labels_	cntr, u, u0, d, jm, p, fpc

m (fuzziness)

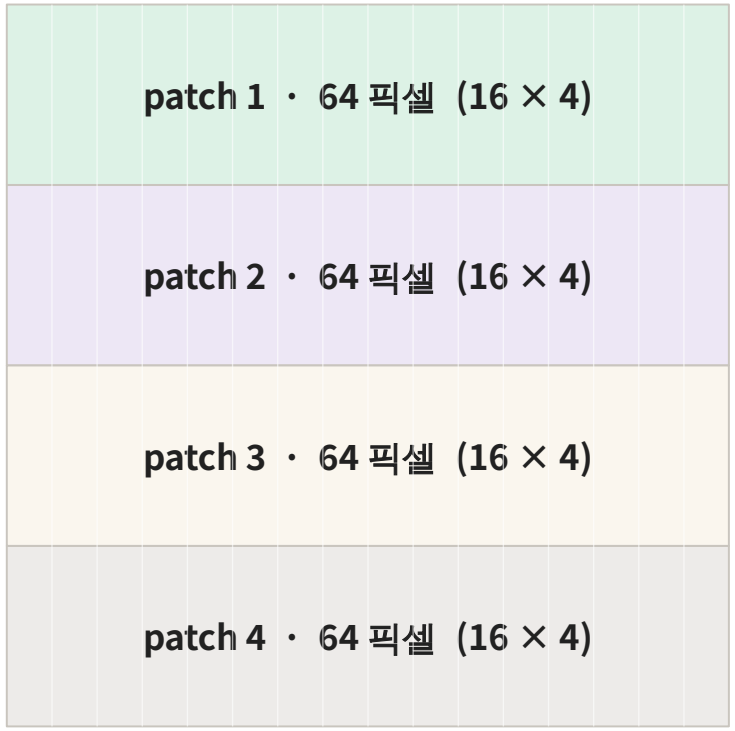
- m → 1: K-means 처럼 딱 잘라 나뉨
- m 이 클수록 소속이 여러 cluster 로 퍼짐
- 보통 2 를 씀 (이 코드도 2)

fpc (0 ~ 1)

분할 선명도 . 1 에 가까울수록 명확하고 , 1/c(여기서는 0.25) 에 가까울수록 모호함

왜 16×16 이고, 어떻게 줄이는가

픽셀 수가 큐빗 수를 정함 - 256 픽셀 = sub-generator 4 개 × 2^6



16 × 16 = 256 픽셀

큐빗 수 계산

256 ÷ 4 = 64 픽셀 /patch → $\log_2 64 = 6$ 데이터 큐빗 → + ancilla 1 = 7 큐빗

보간법	방식	16×16 축소에서
INTER_AREA	출력 픽셀이 덮는 영역의 평균	사용 - 저장소 자산과 같은 조건
INTER_LINEAR	주변 2×2 선형 보간 (기본값)	큰 축소에서 앨리어싱
INTER_NEAREST	가장 가까운 픽셀 1 개 복사	마스크용, 밝기 정보 손실

PART 2 · 노트북 §4 직전

양자 회로 - sub-generator 하나

노이즈 encoding → 변분층 반복 → 측정 → post-selection

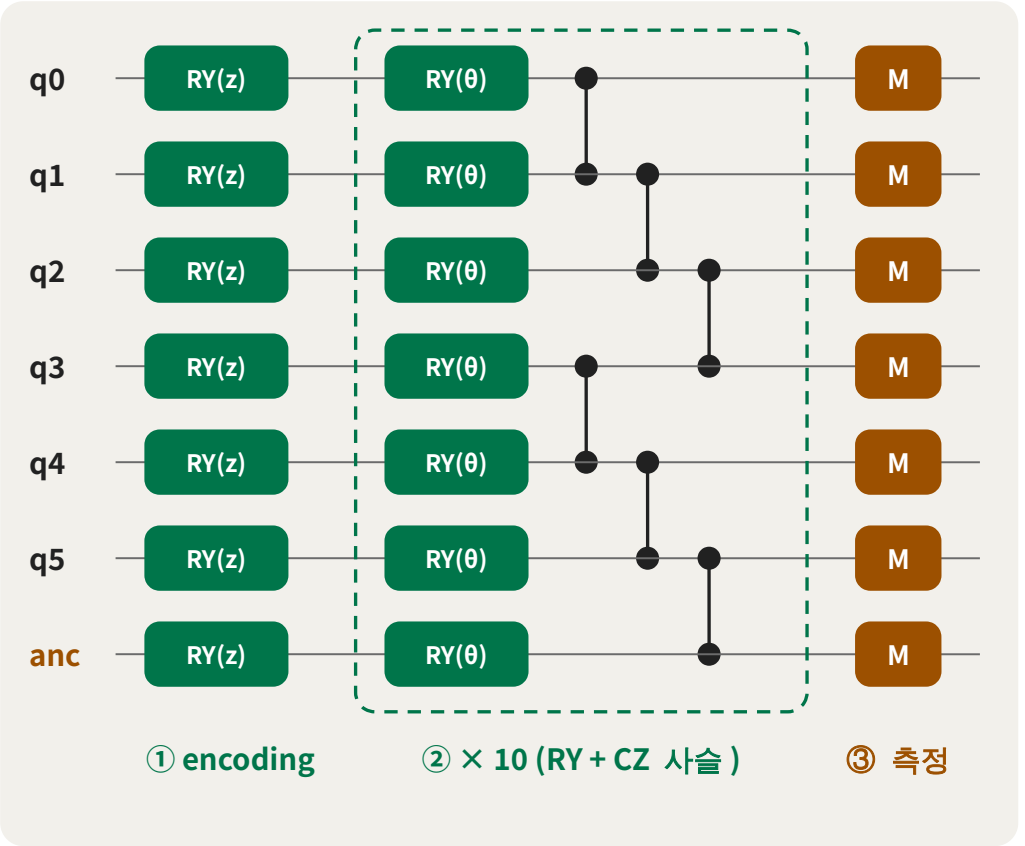
이 파트의 코드	sub_generator_circuit · probs_all[::2] · PatchQuantumGenerator
핵심 질문	회로 출력이 왜 이미지가 되는가 / ancilla qubit 은 왜 두는가 / 파라미터는 몇 개인가

sub-generator 회로

RY 로 노이즈를 넣고, [학습 RY + 인접 CZ] 를 10 번 반복한 뒤 7 큐빗 전체를 측정

```
@qml.qnode(dev)
def sub_generator_circuit(noise, weights):
    for i in range(N_QUBITS):           # 1 encoding
        qml.RY(noise[i], wires=i)
    for layer in range(Q_DEPTH):        # 2 x 10
        for i in range(N_QUBITS):
            qml.RY(weights[layer, i], wires=i)
        for i in range(N_QUBITS - 1):
            qml.CZ(wires=[i, i + 1])
    return qml.probs(wires=range(N_QUBITS))  # 3
```

weights 모양 (10, 7) = 학습 파라미터 70 개 / sub-generator



ancilla qubit 과 post-selection

128 개 확률 중 ancilla qubit = 0 인 64 개만 patch 로 씀 - 코드로는 probs_all[::2]

인덱스	q0 … q5 (데이터)	anc	사용
0	000000	0	**patch 픽셀 0**
1	000000	1	버림
2	000001	0	**patch 픽셀 1**
3	000001	1	버림
…	…	…	…
126	111111	0	**patch 픽셀 63**
127	111111	1	버림

```
probs_all = sub_generator_circuit(noise, weights)
output = probs_all[::2]   # even index = anc 0
```

ancilla 없이 측정하면

- 64 개 확률의 합 = 항상 1
- 배경 patch(합 ≈ 0) 와 병변 patch(합이 큼) 를 구분해 표현할 수 없음

ancilla = 0 만 선택하면

- 64 개 확률의 합 = 0 ~ 1 사이에서 자유로움
- patch 별 밝기 총합도 학습 대상이 됨
- 회로는 그대로 , 측정 · 해석만 바꾼 것

PatchQuantumGenerator - 4 개를 묶어 한 장으로

sub-generator 별로 batch 크기만큼 회로를 실행하고 , patch 를 가로로 이어붙임

```
class PatchQuantumGenerator(nn.Module):
    def __init__(self, n_generators, n_data_qubits, ...):
        q_dev = qml.device("default.qubit", ...)
        @qml.qnode(q_dev, interface="torch",
                   diff_method="backprop")
        def circuit(noise, weights): ...   # same as §4
        self.q_params = nn.ParameterList([
            nn.Parameter(torch.rand(q_depth, n_qubits) * pi)
            for _ in range(n_generators)])

    def forward(self, batch_size):
        patches = []
        for g in range(self.n_generators):      # 4
            rows = [self.circuit(torch.rand(7) * pi,
                     self.q_params[g])[::2]
                    for _ in range(batch_size)] # 8
            patches.append(torch.stack(rows))
        return torch.cat(patches, dim=1)   # (batch, 256)
```

gen(8) 한 번의 실행

sub-gen 1	× 8 샘플	(8, 64)
sub-gen 2	× 8 샘플	(8, 64)
sub-gen 3	× 8 샘플	(8, 64)
sub-gen 4	× 8 샘플	(8, 64)

torch.cat(dim=1) → (8, 256)

회로 실행 4 × 8 = 32 번

파라미터 수 - 생성자 280 개 , 판별자 19,201 개

약 70 배 차이가 나서 학습률을 다르게 둠 (생성자 0.05, 판별자 0.001)

280

생성자 (양자 회전각)
4 × 10 × 7

19,201

판별자 (고전 FC, Table 4)
5 층

판별자 층	모양	파라미터
Linear 1	256 → 64	16,448
Linear 2	64 → 32	2,080
Linear 3	32 → 16	528
Linear 4	16 → 8	136
Linear 5 + Sigmoid	8 → 1	9
합계		**19,201**

학습률

```
opt_g: lr=0.05
opt_d: lr=0.001
```

작은 생성자와 큰 판별자가 비슷한 속도로 배우도록 학습률로 균형을 맞춤

PART 3 · 노트북 §5 학습 셀 실행 후

QGAN 학습

학습 셀을 먼저 실행해 두고 진행 - Colab 에서 400 step 약 35 분

이 파트의 코드	학습 루프 (판별자 step · 생성자 step) · BCE 손실
핵심 질문	로그 숫자를 어떻게 읽는가 / 왜 이렇게 느린가 / 이 설정의 한계는 무엇인가

학습 루프 - 한 step 에 두 번 갱신

판별자를 먼저 갱신하고 , 같은 step 에서 생성자를 갱신함

```
for step in range(STEPS):                          # 400
    batch = real_flat[torch.randint(0, 115, (8,))]

    # discriminator: real -> 1, fake -> 0
    opt_d.zero_grad()
    loss_d = bce(disc(batch), ones) \
           + bce(disc(gen(8).detach()), zeros)
    loss_d.backward(); opt_d.step()

    # generator: make disc say 1 for fakes
    opt_g.zero_grad()
    loss_g = bce(disc(gen(8)), ones)
    loss_g.backward(); opt_g.step()
```

1 **판별자 step**

실제 → 1, 생성 → 0. detach 로 생성자 기울기를 계산하지 않음

2 **생성자 step**

생성 이미지에 라벨 1 - 판별자를 속이는 방향으로 θ 갱신

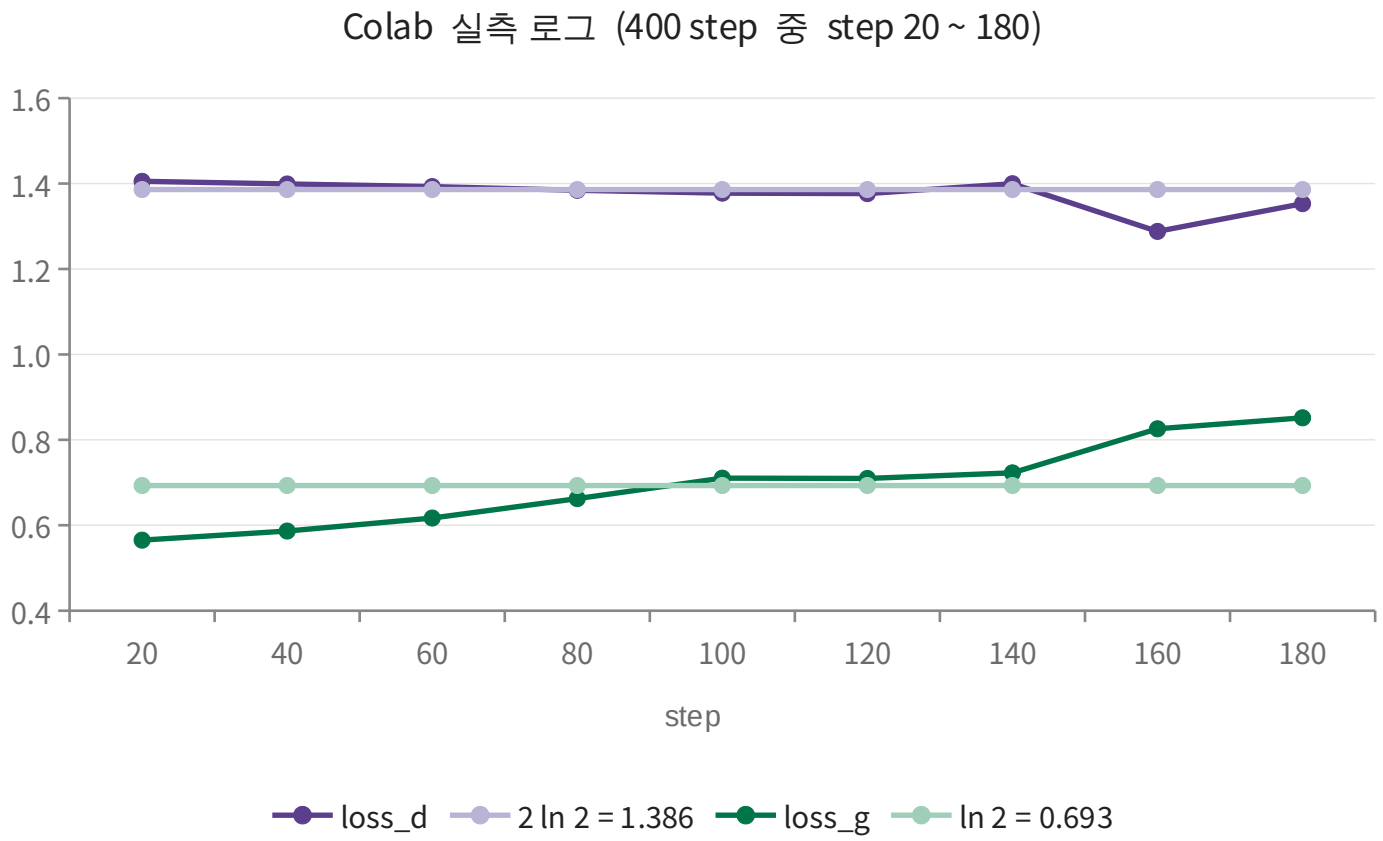
3 **gen() 호출 2 번**

판별자용 1 번 + 생성자용 1 번 → step 당 회로 64 번 실행

PART 3 · 진행 로그 읽기

손실 기준값 - 1.386 과 0.693

판별자가 진짜 · 가짜를 구분하지 못할 때 (D = 0.5) 의 BCE 값이 기준선



값	의미
loss_d ≈ 1.386	구분 불가 - 경쟁 유지
loss_d → 0	판별자 완승 , 생성자 신호 소실
loss_g ≈ 0.693	생성 이미지를 반반으로 판정
loss_g = 0.85	D(G(z)) ≈ e^−0.85 ≈ 0.43

주의 GAN 손실은 단조 감소하지 않음 . 손실이 균형처럼 보여도 품질은 낮을 수 있어 §6 지표로 따로 평가함

PART 3 · 계산 비용

왜 35 분이나 걸리는가

회로 하나는 작지만 ($2^7 = 128$ 차원) 실행 횟수가 많음

32

gen(8) 1 회 회로 실행

4 sub-generator × 8 샘플

64

step 당 회로 실행

판별자 step + 생성자 step

25,600

400 step 전체

7 큐빗 상태벡터 시뮬레이션

미분 방식	어디서	샘플 하나의 생성자 기울기 비용	사용처
backprop	시뮬레이터만	forward 1 번 + 역전파 (상태벡터 직접 미분)	2 일차 QGAN (default.qubit)
adjoint	시뮬레이터만	forward 1 번 + 역방향 1 번, 메모리 적음	3 일차 MediQ-GAN (lightning)
parameter-shift	하드웨어 가능	파라미터당 2 번 → 280 × 2 = 560 번 (× shot 수)	실제 양자 장치

한계 - 실제와 생성 이미지의 픽셀 합이 다름

정규화 단위가 달라 , 판별자가 형태가 아닌 밝기 총합만으로도 구분할 수 있음

	정규화 단위	이미지 한 장의 픽셀 합
실제 이미지 (real_flat)	이미지 전체	항상 1
생성 이미지 (gen(...))	patch 단위 (ancilla = 0 확률)	0 ~ 4 (Colab 실측 약 1.63)

```
real_flat = real_flat / real_flat.sum(dim=1, keepdim=True)   # sum = 1
fake      = gen(8)            # each of 4 patches sums to 0..1
```

1 **실제 이미지도 patch 별로 정규화**

patch 마다 합 1 - 생성 쪽과 단위를 맞춤

2 **생성 patch 를 최댓값으로 나눔**

0 ~ 1 척도로 맞추고 실제 이미지는 그대로 (PennyLane 튜토리얼 방식)

PART 4 · 노트북 §6 직전

생성과 평가

눈으로 보는 것만으로는 판단하지 않음 - 같은 지표 4 종을 3 일차 MediQ-GAN 에도 그대로 씀

이 파트의 코드	gm.to_unit_scale · mean_image_correlation · nearest_neighbor_distance · sample_diversity
핵심 질문	척도를 왜 맞추는가 / 각 지표가 무엇을 보장하고 무엇을 보장하지 않는가

생성품질 지표 4종

16×16 저해상도라 FID 대신 해석 가능한 지표를 씀 (practice/gen_metrics.py)

지표	보는 것	함수	읽는 법
① 픽셀 강도 분포	밝기 분포가 실제와 맞는가	plot_pixel_histogram	척도 변환 전 값도 함께 봄
② 클래스 평균 상관	병변의 전체 형태를 재현했는가	mean_image_correlation	높을수록 좋음 . DF > NV 여야 함
③ 최근접 실제 거리	학습 데이터를 외웠는가	nearest_neighbor_distance	0 에 가까우면 암기
④ 샘플 간 거리	같은 그림만 반복하는가	sample_diversity	0 에 가까우면 mode collapse

to_unit_scale(x, mode) - 세 척도를 0 ~ 1 로

prob — QGAN 출력 이미지별 min-max

tanh — MediQ-GAN 출력 (x + 1) / 2

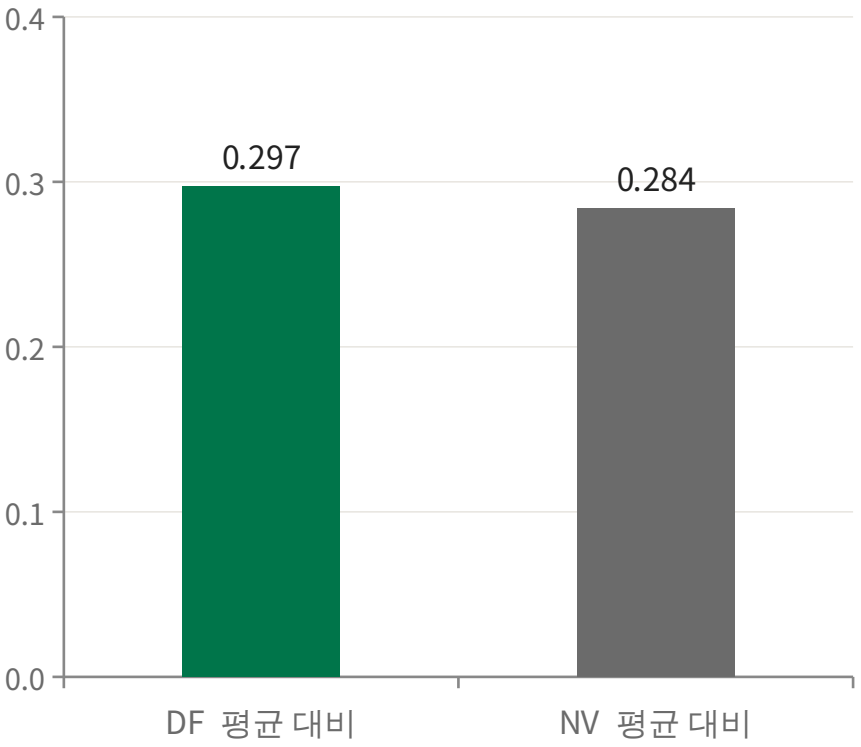
unit — 전처리 데이터 그대로 (0 ~ 1)

PART 4 · 결과 읽기

Colab 실행 결과 - 형태는 잡았지만 클래스 구분은 약함

400 step, 생성 64 장 기준 . 실행마다 새 난수를 쓰므로 소수점보다 격차를 봄

② 클래스 평균 상관계수



1.579

③ 최근접 실제 거리 평균

최솟값 0.956 - 암기 아님

1.731

④ 샘플 간 거리 평균

실제 DF 2.505 의 약 70% - mode collapse 아님

토론 DF 와 NV 상관이 거의 같음 - 생성자가 'DF 다운 ' 특징을 잡았는가 , 아니면 두 클래스의 평균 형태가 원래 비슷한가

PART 4 · 정리

확인 질문

답은 발표자 노트에 있음

1. sub-generator 를 16 개 (논문 설정) 로 늘리고 batch 를 8 로 유지하면 step 당 회로 실행 수는 몇 번인가
2. parameter-shift 로 sub-generator 하나 (70 개 파라미터) 의 기울기를 구할 때 회로 실행 수는 몇 번인가
3. ancilla qubit 없이 데이터 큐빗만 측정하면 어떤 이미지를 만들 수 없게 되는가
4. 클래스 평균 상관계수가 높은데도 좋은 생성모델이 아닐 수 있는 경우는 언제인가
5. 실제 이미지와 생성 이미지의 픽셀 합이 다르면 판별자 학습에 어떤 영향이 있는가

정리

오늘 만든 것과 3 일차

2 일차 결과는 3 일차 비교의 기준선이 됨

오늘 만든 것	segmentation 전처리 · sub-generator 회로 · QGAN 학습 · 지표 4 종 표
저장	qgan_my_16px.pt (생성자 가중치 + 설정 + 손실 기록) → Google Drive
3 일차	MediQ-GAN(layer 단위 hybrid) 미세조정 → 오늘과 같은 지표로 QGAN 과 비교
남은 한계	픽셀 합 불일치 · 16×16 해상도 · exact 시뮬레이션 (shot noise 없음)